

CSCE 641 COMPUTER GRAPHICS · MODULE 02 · LECTURE NOTES

Vectors, Frames, and Homogeneous Coordinates

The algebra the whole pipeline is written in.

MODULE OUTCOMES

Distinguish points from vectors, and explain why the distinction is not pedantry.

Build an orthonormal frame from an arbitrary direction.

Explain what the fourth coordinate means and why translation requires it.

Use dot and cross products fluently for projection, area, and orientation.

Transform normals correctly, and say why the obvious method is wrong.

1 Points and vectors

Both are written as three numbers, which is why the distinction gets lost, and why losing it produces bugs that look like art problems rather than code problems. A **point** is a location: it answers *where*. A **vector** is a displacement: it answers *how far and in which direction*. A vector has no location at all — the same vector exists identically everywhere in space.

The operations that are legal follow directly:

Expression	Result	Meaning
point – point	vector	The displacement from one to the other.
point + vector	point	Move a location by a displacement.
vector ± vector	vector	Compose displacements.
scalar × vector	vector	Scale a displacement.
point + point	undefined	There is no meaningful sum of two locations. (A weighted average with weights summing to 1 is meaningful — that is exactly what barycentric interpolation is, and Module 04 relies on it.)

THE PRACTICAL CONSEQUENCE

Translation must affect points and must not affect vectors. A surface normal pointing up still points up after you move the object three metres to the left.

Homogeneous coordinates make this automatic rather than something you have to remember.

2 Homogeneous coordinates

Add a fourth coordinate w . Points get $w = 1$, directions get $w = 0$. Now consider a translation matrix:

$$\begin{bmatrix} 1 & 0 & 0 & tx \\ 0 & 1 & 0 & ty \\ 0 & 0 & 1 & tz \\ 0 & 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix} = \begin{bmatrix} x + tx*w \\ y + ty*w \\ z + tz*w \\ w \end{bmatrix}$$

With $w = 1$ the translation applies. With $w = 0$ it vanishes. The same matrix does the correct thing to both types without a branch.

That alone would be a convenience. The reason homogeneous coordinates are *necessary* rather than convenient is the equivalence relation they carry:

$$(x, y, z, w) \equiv (\lambda x, \lambda y, \lambda z, \lambda w) \quad \text{for any } \lambda \neq 0$$

All of these name the same point, recovered as $(x/w, y/w, z/w)$. This is what lets perspective projection — which is a division, and therefore not linear — be expressed as a linear matrix followed by a single divide at the end. The matrix arranges for w to come out equal to the view-space depth; the divide then produces the $1/z$ falloff that makes distant things small. Module 03 does this in full.

POINTS AT INFINITY

A direction is a point with $w = 0$ — literally a point infinitely far away in that direction. This is not a metaphor; it is how projective geometry works, and it is why directional lights (the sun) are naturally expressed as $w = 0$ while point lights use $w = 1$.

It also explains why parallel lines meet at the horizon in a perspective image: the horizon is where the $w = 0$ points land.

3 The two products

3.1 Dot product

$$\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y + a_z b_z = |\mathbf{a}| |\mathbf{b}| \cos \theta$$

Two formulas for the same number: the first computes it, the second explains it. For unit vectors the dot product is the cosine of the angle between them, which is why so much of shading is clamped dot products.

The sign alone answers 'are these pointing roughly the same way?', which is the whole of backface culling.

Projection of **a** onto a *unit* direction **b** is $(\mathbf{a} \cdot \mathbf{b}) \mathbf{b}$. Subtracting that projection leaves the perpendicular component. Splitting a vector into parallel and perpendicular parts this way is the core operation of Gram–Schmidt orthogonalisation, of collision response, and of reflecting a vector about a normal:

$$\text{reflect}(\mathbf{d}, \mathbf{n}) = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{n}) \mathbf{n}$$

3.2 Cross product

Produces a vector perpendicular to both inputs, with length equal to the area of the parallelogram they span. Both halves are used constantly: the direction gives you normals and frames, the magnitude gives you areas, and the *sign* of the area in 2D gives you orientation — which is how Module 04 decides whether a point is inside a triangle.

The cross product is anticommutative: $\mathbf{a} \times \mathbf{b} = -(\mathbf{b} \times \mathbf{a})$. This is the entire content of 'winding order'. A triangle's normal depends on the order its vertices are listed in, and a renderer that disagrees with its asset pipeline about that convention renders everything inside out.

4 Coordinate frames

A frame is an origin and three basis vectors. A coordinate triple (x, y, z) means 'start at the origin, go x along the first basis vector, y along the second, z along the third'. Changing frames is re-expressing the same geometric object in different numbers, and the whole vertex pipeline is a sequence of exactly this.

A frame whose basis vectors are mutually perpendicular and unit length is **orthonormal**, and orthonormal matrices have a property worth caring about:

$$\mathbf{M}^{-1} = \mathbf{M}^T \quad (\text{orthonormal only})$$

Inverting becomes transposing — nine memory moves instead of a general 4×4 inverse. This is precisely why the view matrix is cheap to build: the camera's orientation is orthonormal, so its inverse is free, and only the translation needs real work. Module 03 builds the view matrix on exactly this observation.

4.1 Building a frame from one vector

Tangent-space normal mapping (Module 08) and hemisphere sampling (Module 11) both need a full orthonormal frame given only a normal. The construction is three lines, and the guard is mandatory:

```
void make_frame(const vec3& n, vec3& t, vec3& b) {
    // If `up` is nearly parallel to n, cross(up, n) -> 0 and
    // normalize() returns NaN. This shows up as a few black or white
    // pixels that move with the camera -- almost impossible to find
    // if you do not already know to look for it.
    vec3 up = (fabs(n.z) < 0.999f) ? vec3(0,0,1) : vec3(1,0,0);
    t = normalize(cross(up, n));
    b = cross(n, t);
}
```

Note that **b** needs no normalization: the cross product of two perpendicular unit vectors is already unit length. Small thing, but it runs per-fragment.

5 Transforming normals correctly

This is the single most commonly botched piece of linear algebra in real-time graphics, and the symptom — lighting that is slightly wrong on stretched objects — is easy to blame on the artist.

5.1 Why the obvious method fails

Take a sphere and scale it by 2 along X, flattening it in Y and Z relatively. Consider a point on the surface at 45°. The surface there becomes *shallower*, so its normal should rotate *toward* the Y axis. But applying the same scale to the normal stretches its X component, rotating it toward X — the opposite direction. The transformed vector is no longer perpendicular to the transformed surface.

5.2 The derivation

Let $\mathbf{t}$ be any tangent to the surface, so $\mathbf{n} \cdot \mathbf{t} = 0$. After transformation by $\mathbf{M}$ the tangent becomes $\mathbf{M}\mathbf{t}$, and we want some matrix $\mathbf{G}$ such that the transformed normal $\mathbf{G}\mathbf{n}$ is still perpendicular to it:

$$(\mathbf{G}\mathbf{n}) \cdot (\mathbf{M}\mathbf{t}) = 0 \quad \Rightarrow \quad (\mathbf{G}\mathbf{n})^T(\mathbf{M}\mathbf{t}) = 0 \quad \Rightarrow \quad \mathbf{n}^T(\mathbf{G}^T\mathbf{M})\mathbf{t} = 0$$

We know $\mathbf{n}^T\mathbf{t} = 0$ holds for every tangent $\mathbf{t}$. So the equation is satisfied for all $\mathbf{t}$ precisely when $\mathbf{G}^T\mathbf{M} = \mathbf{I}$, which gives:

$$\mathbf{G} = (\mathbf{M}^{-1})^T$$

This matrix is called the **normal matrix**. Two useful special cases: if $\mathbf{M}$ is a pure rotation then $\mathbf{M}$ is orthonormal and $(\mathbf{M}^{-1})^T = \mathbf{M}$, so nothing changes. If $\mathbf{M}$ is a uniform scale then $(\mathbf{M}^{-1})^T$ is proportional to $\mathbf{M}$, which changes only length — and you normalize anyway. The inverse transpose therefore matters exactly when there is non-uniform scale or shear.

TWO SEPARATE REQUIREMENTS

- 1. Use the normal matrix.** Compute it once per object on the CPU; a 3×3 inverse per vertex would be absurd.
- 2. Renormalize after interpolation.** The rasterizer interpolates normals linearly across the triangle, and the linear interpolation of two unit vectors is not a unit vector — it is shorter, most so in the middle of the triangle. Skipping this darkens the centre of every large triangle, which looks exactly like a bad lightmap. Both are needed. Fixing one and not the other leaves you with a subtler version of the same bug.

5.3 The cheat that is usually correct

Many engines upload only a 3×3 rotation and skip the inverse transpose, on the grounds that scene objects rarely carry non-uniform scale. This is a defensible engineering decision as long as it is *decided* rather than overlooked. If you take it, assert on non-uniform scale in your asset pipeline so that the day someone squashes a prop you get an error rather than a mystery.

6 Reference table

Quantity	w	Transform by	Post-step
Position	1	M	Divide by w after projection.
Direction, tangent, bitangent	0	M	Normalize if length matters.
Normal	0	$(M^{-1})^T$	Always renormalize, including after interpolation.
Plane (n_x, n_y, n_z, d)	—	$(M^{-1})^T$ applied to the full 4-vector	Renormalize n and rescale d consistently.
Ray direction	0	M	Do not normalize before intersection tests if the test assumes parametric t in object units.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
3Blue1Brown — Essence of Linear Algebra	Chapters 1–9. The best available geometric intuition for bases, determinants, and change of basis. Watch before this module if linear algebra feels like symbol-pushing.
GAMES101 Lecture 02 — Review of Linear Algebra	The same material aimed specifically at graphics, including the geometric meaning of both products.
Scratchapixel — Geometry	Written reference with the normal-matrix derivation in full.
Immersive Math — interactive linear algebra	Manipulable figures. Useful for building intuition about frames and projections.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement a `vec3`, `vec4`, and `mat4` with the standard operations. Write them yourself even though GLM exists — you will reason about the pipeline better for having done it once.
- 2 Write a unit test that constructs a non-uniform scale, transforms a normal with M and with $(M^{-1})^T$, and asserts that only the second remains perpendicular to a transformed tangent. This test will catch the bug permanently.

- 3 Implement `make_frame` and verify numerically, over ten thousand random normals, that the resulting basis is orthonormal to within floating-point tolerance and never produces NaN. Deliberately remove the guard and find the inputs that break it.
- 4 Given a triangle's three vertices, compute its face normal and its area using one cross product. Confirm that reversing two vertices negates the normal and leaves the area magnitude unchanged.
- 5 Take two unit vectors 60° apart, interpolate linearly halfway between them, and measure the result's length. Explain in one sentence why interpolated normals darken the middle of large triangles if you do not renormalize.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is `point + point` undefined, while a weighted average of two points with weights summing to one is perfectly well defined?
- 2 State the equivalence relation on homogeneous coordinates, and explain why it is what makes perspective projection expressible as a matrix.
- 3 What does $w = 0$ mean geometrically? Why is a directional light naturally $w = 0$?
- 4 Derive the normal matrix from the requirement that $\mathbf{n} \cdot \mathbf{t} = 0$ be preserved.
- 5 Under what transformations is the inverse transpose unnecessary, and why?
- 6 Give two distinct reasons a normal might need renormalizing in the fragment shader, and say what each one looks like if you skip it.